THEORY OF COMPUTATION
&
COMPUTER ORGANIZATION & ARCHITECTURE

Combined Section-wise Study Notes

Part A — TOC: strings & languages · grammars & the Chomsky hierarchy · finite automata (DFA, NFA, ε-NFA, Moore, Mealy) · regular expressions · pushdown automata · Turing machines

Part B — COA: organization vs architecture · von Neumann & Harvard · ISA (CISC, RISC) · pipeline & microarchitecture · error detection & correction · buses · microprocessor & microcontroller

Typed from the two sets of handwritten class notes — every topic, example and diagram retained, with clarifications where a page was unclear or needs a correction.

Contents

A1. Introduction — Symbols, Strings and Languages 5

A1.1 Basic terms . . . 5

A1.2 Language — finite and infinite . . . 5

A1.3 Prefix and suffix of a string . . . 5

A1.4 Kleene closure (Kleene star) . . . 5

A2. Grammar and the Chomsky Hierarchy 7

A2.1 Grammar . . . 7

A2.2 Chomsky hierarchy . . . 7

A3. Machines and Finite Automata 9

A3.1 The machines at a glance . . . 9

A3.2 Kinds of finite automata . . . 9

A3.3 DFA — Deterministic Finite Automaton . . . 9

A3.4 DFA vs NFA . . . 10

A3.5 ε-NFA . . . 10

A3.6 Moore vs Mealy machine . . . 10

A4. Regular Expressions 12

Precedence (highest to lowest) . . . 12

A5. Turing Machine 12

A5.1 Structure of a Turing machine . . . 12

A5.2 ID — Instantaneous Description . . . 13

A5.3 Language accepted by a Turing machine . . . 13

A5.4 Variants of the Turing machine . . . 13

A5.5 Universal Turing machine . . . 13

A5.6 Church’s thesis . . . 13

A6. Pushdown Automata (PDA) 13

A6.1 Working of a PDA . . . 14

A6.2 DPDA vs NPDA . . . 14

B1. Computer Organization vs Computer Architecture 16

B1.1 Computer organization . . . 16

B1.2 Computer architecture . . . 16

B1.3 Difference . . . 16

B2. Types of Computer Architecture 16

B2.1 Von Neumann architecture . . . 16

B2.2 Harvard architecture . . . 17

B2.3 Modified Harvard architecture . . . 17

B3. ISA — Instruction Set Architecture 19

B3.1 Types of ISA . . . 19
B3.2 Types of instructions in an ISA . . . 19
B4. Pipeline, Microarchitecture and System Design 19
B4.1 Pipeline . . . 19
B4.2 Microarchitecture . . . 20
B4.3 System design architecture . . . 20
B5. Error Detection and Correction Codes 21
B5.1 Types of errors . . . 21
B5.2 Error detection codes . . . 21
B5.3 Error correction code . . . 21
B5.4 Comparison — parity vs checksum vs Hamming . . . 23
B6. Bus 24
B7. Microprocessor and Microcontroller 24
B7.1 Microprocessor . . . 24
B7.2 Microcontroller . . . 25
Appendix — Clarifications and reading notes 26

PART A

Theory of Computation

Automata · Grammars · Languages · Machines

A1. Introduction — Symbols, Strings and Languages

- **Automata** = the study of **abstract computing devices or machines**.

A1.1 Basic terms

Term	Meaning	Examples
Symbol	An abstract entity	a, b, 0, 1 …
Alphabet (Σ)	A **finite set / collection of symbols**	Σ = {a, b, c}; Σ = {0, 1}
String	A **sequence of symbols**	Σ = {a, b}: a, b, aa, ab, ba, bb … (aa, ab, ba, bb have length 2)
Empty / null string	A string whose length is **zero**; denoted by **ε** or **λ**	ε
Language	A **collection of strings**, restricted over a given alphabet	see below
Length of a string	The **number of symbols** in the string	|ab| = 2; |ε| = 0

A1.2 Language — finite and infinite

Example: Σ = {a, b}

Language	Defined as	Strings	Size
L1	set of all strings of length 2	{aa, ab, ba, bb}	**finite** (4)
L2	set of all strings that start with “a”	{a, ab, aa, aab, aaa, …}	**infinite**

If the length of the strings is given, the language is finite; otherwise it can be infinite.

A1.3 Prefix and suffix of a string

- **Prefix**: if w = xy for some string y, then **x is a prefix** of w.
- **Suffix**: if w = xy for some string x, then **y is a suffix** of w.

Example: w = 0011

Split x | y	Prefix x	Suffix y
0 | 011	0	011
00 | 11	00	11
001 | 1	001	1

All prefixes of 0011: ε, 0, 00, 001, 0011. All suffixes: ε, 1, 11, 011, 0011 (the complete lists are added; the notes show the three splits).

A1.4 Kleene closure (Kleene star)

- Denoted by * (asterisk) after the name of the alphabet, i.e. **Σ***; the notation is also called the **Kleene star**.
- Σ^n = set of all strings of length n; $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$
- $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$ — the set of all strings of **all lengths** over the alphabet.

Example: Σ = {a, b}

n	Σ^n	Strings	Count = 2^n
0	Σ^0	{ε}	1
1	Σ^1	{a, b}	2
2	$\Sigma^2 = \Sigma^1 \cdot \Sigma^1$	{aa, ab, ba, bb}	4
3	Σ^3	{aaa, aab, aba, abb, baa, bab, bba, bbb}	$8 = \lvert\Sigma\rvert^3$
n	Σ^n	all strings of length n	2^n

A2. Grammar and the Chomsky Hierarchy

A2.1 Grammar

- A grammar **enumerates the strings of a language**.
- It is a **finite set of rules** for defining a language.
- Defined as a **4-tuple (V, T, P, S)**:

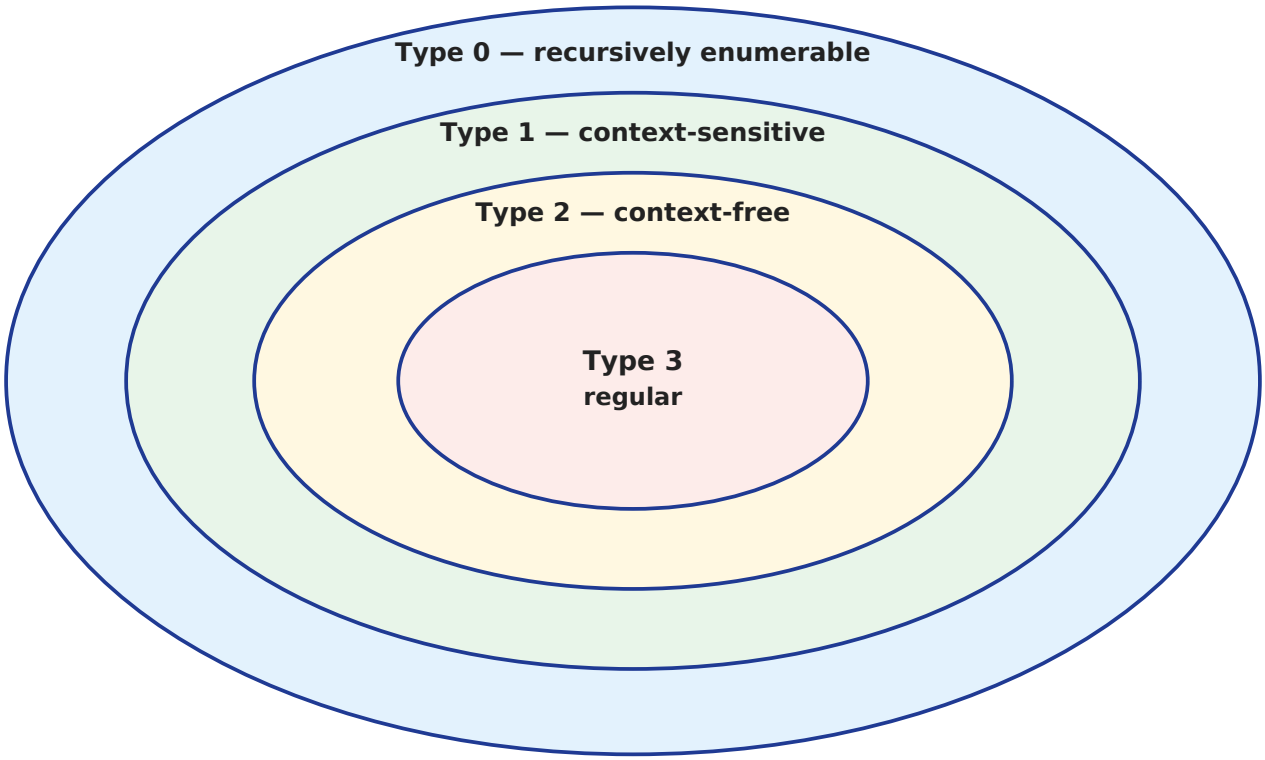
Symbol	Meaning
V	Set of **non-terminals** (variables)
T (Σ)	Set of **input terminals**
P	Finite set of **production rules**
S	**Start symbol**

Remember

Getting a string from a grammar = **derivation**.

A2.2 Chomsky hierarchy

- It consists of **4 classes**: Type 0, Type 1, Type 2, Type 3 — each class is contained in the one before it.



Chomsky hierarchy: the four nested classes with their grammar, language and machine.

Type	Grammar	Language (also known as)	Accepted by
Type 0	Unrestricted grammar	Recursively enumerable language	Turing machine
Type 1	Context-sensitive grammar	Context-sensitive language	Linear bounded automaton (LBA)
Type 2	Context-free grammar	Context-free language	Pushdown automaton (PDA)
Type 3	Regular grammar	Regular language	Finite state automaton (FSA / finite automata)

(A) Type-0 grammar

- The **unrestricted grammar**; includes almost all formal languages.

- Generates exactly those languages that can be recognized by a **Turing machine**.
- Also known as: Turing machine, recursively enumerable language.

(B) Type-1 grammar

- **Context-sensitive** grammar.
- The languages defined by a Type-1 grammar are accepted by **linear bounded automata**.
- Also known as: context-sensitive language, linear bounded automata.

(C) Type-2 grammar

- **Context-free** grammar; accepted by a **PDA**.
- Also known as: context-free language, pushdown automata.

(D) Type-3 grammar

- **Regular** grammar; accepted by a **finite state automaton**.
- Also known as: regular language, finite automata.

> **Clarification**
> In the handwritten picture the innermost ellipse is labelled “Recursive language”; since it belongs to Type 3 with the FSA, it is shown here as **regular language**.

A3. Machines and Finite Automata

A3.1 The machines at a glance

Machine	Memory	Accepts	Applications
Finite automaton	Fixed amount of **unstructured** memory	Regular languages	Modelling state, communication protocols, adventure games, some control systems
PDA	**Unbounded structured** memory — a push-down stack	Context-free languages	Modelling parsing, compilers, programming-language design
Turing machine	**Unbounded tape**	Recursively enumerable languages	Model for general sequence computing; equivalent to RAM and programming-language models

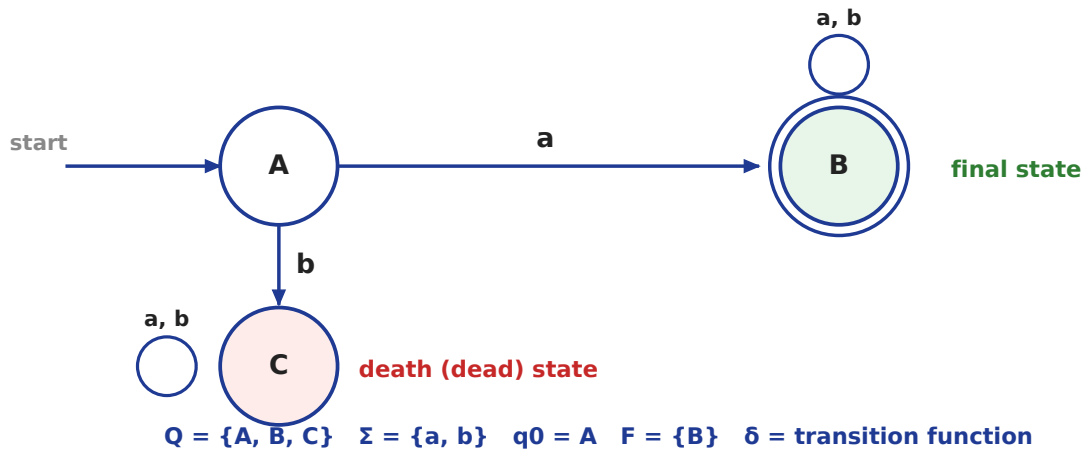
A3.2 Kinds of finite automata

Without output	With output
DFA — deterministic finite automaton NFA — non-deterministic finite automaton ε-NFA	(1) Moore machine (2) Mealy machine

A3.3 DFA — Deterministic Finite Automaton

- A finite automaton is defined as a **5-tuple (Q, Σ, q0, F, δ)**:

Symbol	Meaning	In the example
Q	Set of all states	{A, B, C}
Σ	Inputs	{a, b}
q0	Initial state	A
F	Final state(s)	B
δ	Transition function	see the diagram



DFA from the notes: A is the start state, B (double circle) is the final state, C is the death (dead) state.

From state	on a	on b
→ A (start)	B	C
B (final)	B	B

From state	on a	on b
C (death)	C	C

Reading note

The label on the arrow A → B is not legible in the handwritten diagram; with b leading to the death state, it is read as **a**. The DFA then accepts exactly the strings that **start with a** (the language L2 of section A1.2).

A3.4 DFA vs NFA

#	DFA	NFA
1	Each DFA can be an NFA	Each NFA can't be a DFA (as written)
2	DFA and NFA have the same power; a DFA can be translated to an NFA	—
3	Both can have multiple final states	—
4	A compiler uses a DFA in lexical analysis	A compiler doesn't use an NFA
5	Backtracking is possible (as written)	Not always possible (as written)
6	Requires more space	Requires less space

Clarification

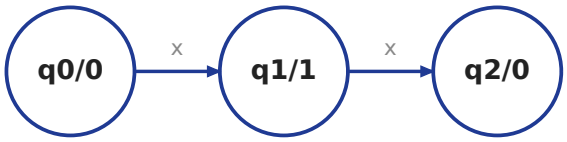
Standard statements: every DFA is also an NFA, and every NFA can be converted into an equivalent DFA (same power, possibly with more states — which matches "DFA requires more space"). Backtracking/guessing belongs to the NFA, while a DFA has exactly one move for each input. The table above is kept as written; use the standard statements in exams.

A3.5 ε-NFA

- In an NFA, moving on **null (ε)** is allowed — it can move **without reading any symbol**.
- Used to identify regular languages.
- This makes the NFA more powerful in convenience, as it can transition without any input.

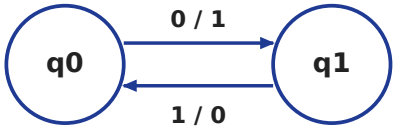
A3.6 Moore vs Mealy machine

Moore — output is attached to the STATE



output depends only on the current state

Mealy — output is attached to the TRANSITION



label = input / output

Moore machine: output written inside the state. Mealy machine: output written on the transition (input / output). Illustration added.

Point	Moore machine	Mealy machine
(1) Basic definition	Output depends **only on the current state**; the output table is associated with the **state**	Output depends on **both the current state and the input**; the output table is associated with the **input**
(2) Structure	Common: state (Q), initial state (q0), output alphabet (Δ), input alphabet (Σ), transition function (δ). Output function λ : **Q → Δ**	Same components. Output function λ : **Q × Σ → Δ**

Point	Moore machine	Mealy machine
(3) Example	If the current state is q1, the output = 1 no matter what the input is	If the state is q1 and input = 0, output = 1; if the state is q1 and input = 1, output = 0
(4) Speed of output	Output is received **after a state change** — slower	Output is received **immediately with the input** — faster
(5) Number of states	**More** states, because the output is linked to states	**Fewer** states, because the output is linked to transitions
(6) Conversion	Moore → Mealy: possible, results in **fewer** states	Mealy → Moore: possible, results in **more** states

Output tables from the notes:

State	Output
q0	0
q1	1
q2	0

Transition	Output
(q1, 0)	1
(q1, 1)	0

Left: Moore machine (output per state). Right: Mealy machine (output per state + input pair).

A4. Regular Expressions

- A **regular expression (RE)** is a method used to **define regular languages**.
- It uses **symbols and operators** to define patterns of strings.

Expression	Meaning
a | b	**a or b** — union in a regular language
a*	**zero or more** occurrences of a

Precedence (highest to lowest)

Rank	Operator
1 (highest)	Kleene star (*)
2	Concatenation
3 (lowest)	Union (|)

Example: Using the precedence

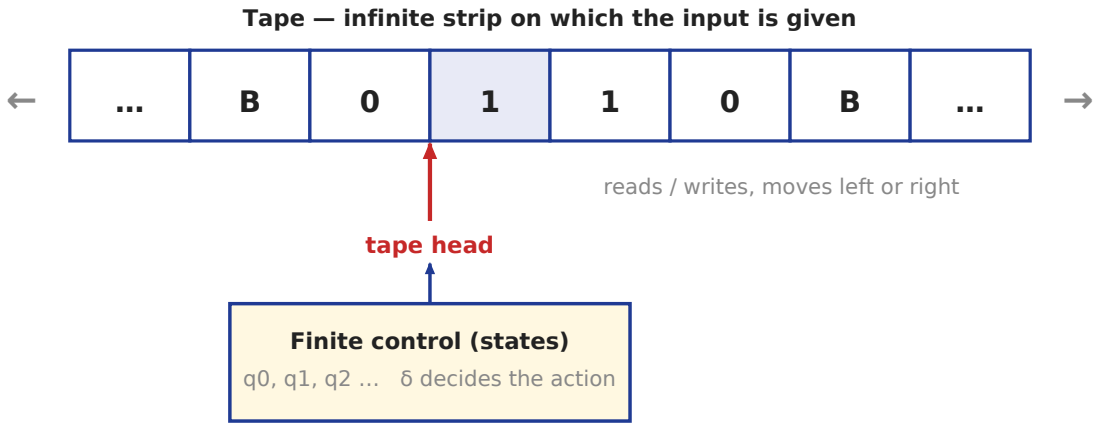
ab*|c is read as (a(b*)) | c — the star applies first (to b), then concatenation (a·b*), then union with c. Strings: a, ab, abb, … and c.

The worked example above is added for illustration; the notes give the operators and the precedence order.

A5. Turing Machine

- A **mathematical model of computation** that can **simulate any algorithm**.
- Invented by **Alan Turing (1936)**.
- A theoretical computation machine which shows that a computer can solve certain problems and not others.

A5.1 Structure of a Turing machine



Turing machine: infinite tape, tape head and finite control (illustration of the four parts listed in the notes).

Part	Meaning
(A) Tape	An **infinite strip** on which the input is given; during computation the machine can read, write (and erase)
(B) Tape head	A **pointer on the current cell**; it can read / write and **move left / right**
(C) Finite control / state	A finite set of states (q0, q1, q2 …) showing the current status of the machine

Part	Meaning
(D) Transition function (δ)	Decides what action will be taken on which input symbol

A5.2 ID — Instantaneous Description

- An ID describes **which state the machine is in**, **what is written on the tape**, and **at which position the head is**.

A5.3 Language accepted by a Turing machine

- A Turing machine accepts a language if, after processing the input string, it **reaches the accept state**.
- **2 modes of acceptance:** (a) acceptance by **final state**; (b) acceptance by **halting state**.
- A TM accepts **recursively enumerable (RE)** or **Type-0** languages.

A5.4 Variants of the Turing machine

Variant	Idea
(i) Multi-tape	Multiple tapes — but equivalent to a single-tape TM
(ii) Multi-track	Multiple tracks on a single tape (parallel)
(iii) Non-deterministic	Multiple choices are possible for one input
(iv) Two-way infinite tape TM	Infinite tape on both sides

Important

All of these variants have **computational power equivalent to the standard Turing machine**.

A5.5 Universal Turing machine

- A TM that can **simulate the behaviour of any other TM**.
- It is given **both input and program** (the code of a TM).
- This forms the basis of the concept of a **general-purpose computer**.

A5.6 Church's thesis

Church's thesis

"Whatever is computable by an **effective method** is computable by a **Turing machine**."

A6. Pushdown Automata (PDA)

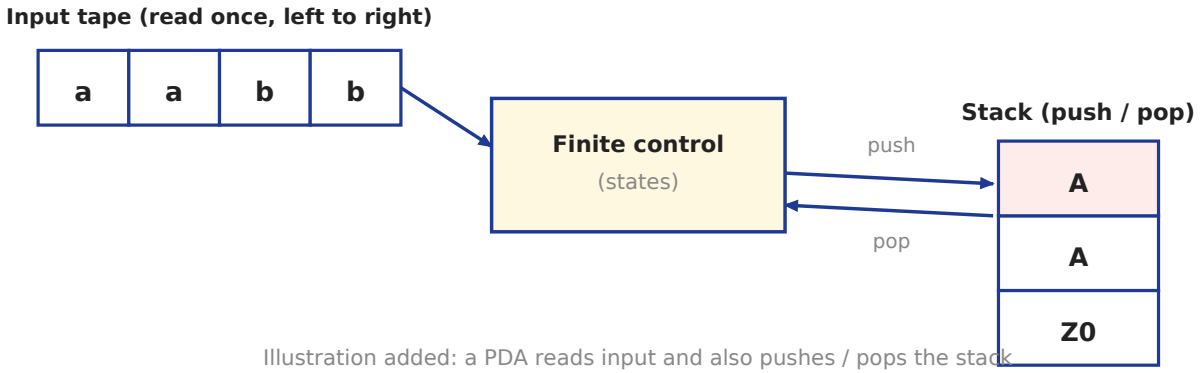


Illustration added: a PDA reads input and also pushes / pops the stack

A PDA = finite control + input + a push-down stack.

Feature	Finite automata	PDA
(1) Memory	No external memory	Stack

Feature	Finite automata	PDA
(2) Power	Less powerful (regular languages)	More powerful (context-free languages)
(3) Example	String ending with “ab”	Balanced parentheses & palindromes

A6.1 Working of a PDA

- When a PDA reads any input, it **also pushes or pops symbols onto / from the stack**.
- A PDA can accept a string in two ways:

Way	Accepts when…
(a) By final state	The input finishes and the machine reaches the **final state**
(b) By empty stack	The input finishes and the **stack becomes empty**

Important

Both methods are **equivalent** — by using either of them the same languages can be recognized.

A6.2 DPDA vs NPDA

	DPDA (deterministic)	NPDA (non-deterministic)
Choices	Only **one possibility** at each step	**More than one** possibility at each step
Power	Accepts **fewer** languages	**More powerful** — accepts more languages

PART B

Computer Organization & Architecture

Architectures · ISA · Pipeline · Error codes · Buses · Processors

B1. Computer Organization vs Computer Architecture

B1.1 Computer organization

- Understanding the **internal working** of a computer.
- Deciding **which hardware will perform which task, and how**.
- It includes: how **data flows** (between CPU / memory / I/O); how **control signals** are generated; the **structure of memory** (RAM, ROM, cache etc.); how the **processor works** and how instructions are executed.

B1.2 Computer architecture

- **How the computer is designed.**
- It includes: **(a) Instruction Set Architecture (ISA) (b) data type (c) register design (d) addressing mode**.

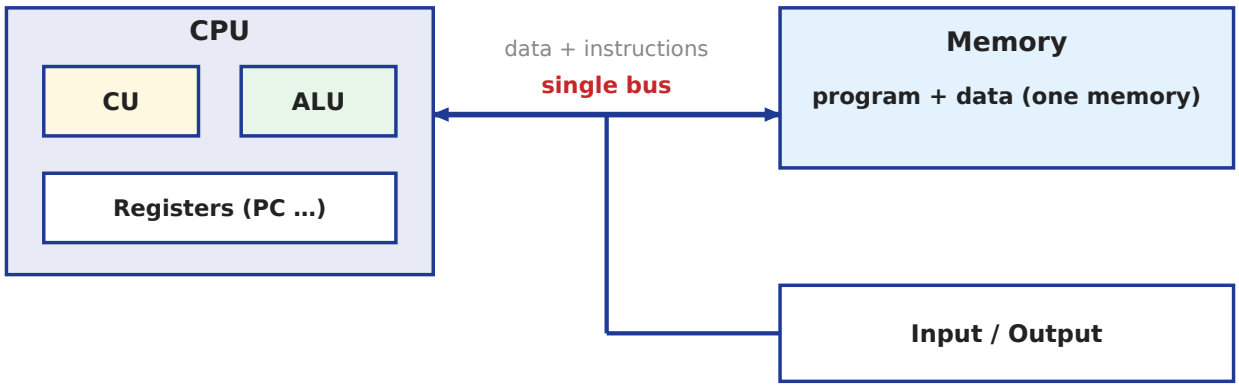
B1.3 Difference

Difference	Organization	Architecture
Focus	Implementation	Design
Related to logic / level (heading hard to read)	No	Yes
Example	Control signals, memory layout	Instruction set, registers
Who works on it / supported by	Hardware engineers, system designers	Software developers, computer designers

B2. Types of Computer Architecture

B2.1 Von Neumann architecture

- The **oldest and most widely used** architecture, in which **data and instructions are accessed from the same memory**.
- Proposed by **John von Neumann in 1945**.



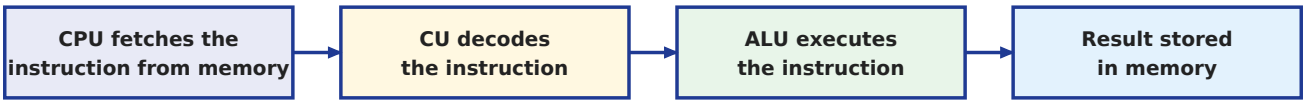
Von Neumann architecture: CPU (CU, ALU, registers), one memory for program and data, and one shared bus (illustration added).

Features

- **Single memory** for program and data.
- Instructions are executed **sequentially**.
- **Simple design**.

- The **program counter** tells the address of instructions.

Working



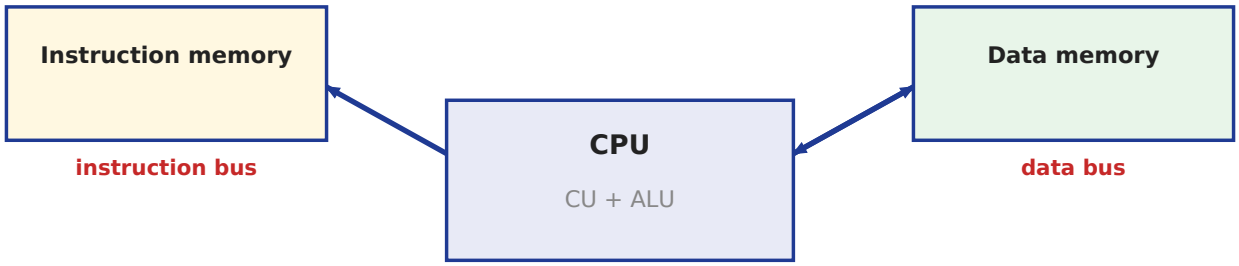
Von Neumann working cycle as described in the notes.

Problem

- Data and instructions **share the same bus**, so the CPU has to wait every time until data and instructions are fetched from memory.
- This causes **performance to slow down**.
- Ex: Intel 8085, 8086.

B2.2 Harvard architecture

- Invented at **Harvard University**.
- Data and instructions have **separate buses and memory**.



separate buses and memories → fetch an instruction and read data at the same time

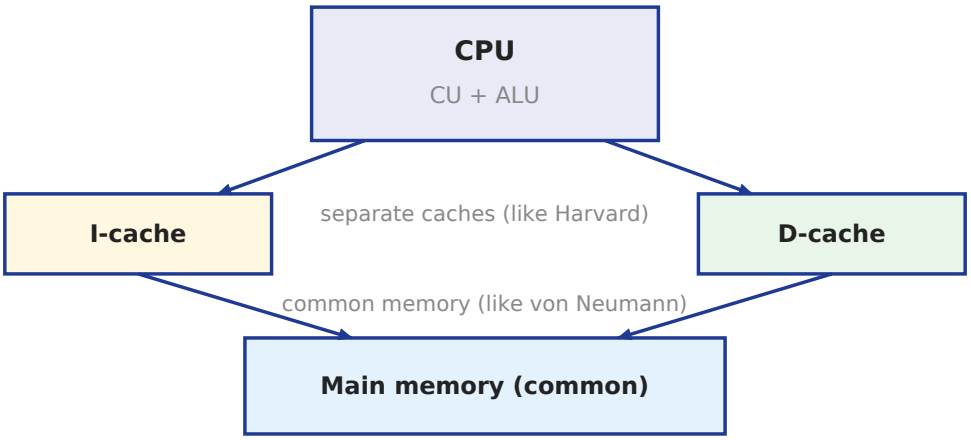
Harvard architecture: separate instruction and data memories with separate buses (illustration added).

Working / features	Drawbacks
The CPU can read data from memory and fetch instructions from memory **at the same time**; since both tasks happen in parallel, performance is **fast**. Separate memory and buses; parallel processing possible.	Design is **complex and costly**. Data and instructions have separate memory space, which **reduces flexibility**.

Examples: ARM Cortex-M series, PIC controller, DSP.

B2.3 Modified Harvard architecture

- In modern CPUs, the features of **both von Neumann and Harvard** architectures are combined.
- Memory is **common** (like von Neumann), but there are **separate caches** for data and instructions (like Harvard).
- Provides **higher speed and an efficient design**.
- Examples: Intel i7, AMD Ryzen, ARM Cortex-A series.



Modified Harvard: separate instruction and data caches in front of one common main memory (illustration added).

	Von Neumann	Harvard	Modified Harvard
Memory	One for program + data	Separate instruction and data memories	Common memory
Buses / caches	Single bus	Separate buses	Separate caches for data and instructions
Speed	Slower (CPU waits)	Fast (parallel)	Higher speed
Design	Simple	Complex and costly	Efficient
Examples	Intel 8085, 8086	ARM Cortex-M, PIC, DSP	Intel i7, AMD Ryzen, ARM Cortex-A

Comparison table added for revision from the three descriptions above.

B3. ISA — Instruction Set Architecture

- The **interface between hardware (CPU) and software (program)**.
- Defines **which instructions the processor can understand and execute**.

B3.1 Types of ISA

	(A) CISC	(B) RISC
Full form	Complex Instruction Set Computing	Reduced Instruction Set Computing
Instructions	Many **complex** instructions; one instruction can perform multiple low-level operations	**Simple, fixed-length** instructions; each executes in **one clock cycle**
Example	Intel x86	ARM
Code size	**Small** — fewer instructions need to be written	Becomes **very large** (main disadvantage)
Speed	**Slow** execution (main disadvantage) due to complex decoding	**Faster** execution; easy to design a **pipeline**

B3.2 Types of instructions in an ISA

#	Type	Instructions
(i)	Data transfer	MOV, LOAD, STORE
(ii)	Arithmetic	ADD, SUB, MUL
(iii)	Logical	AND, OR, NOT
(iv)	Control flow	JMP, CALL, RET
(v)	Input / output	IN, OUT
(vi)	Comparison	CMP

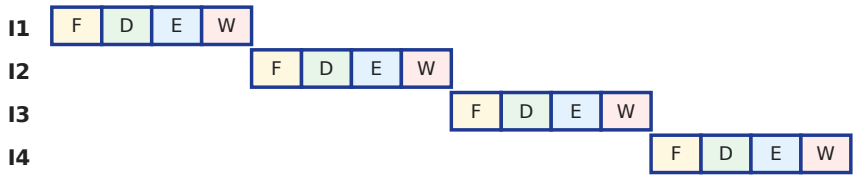
The notes list the logical instructions as “AND, OR, AND”; the third one is read as NOT.

B4. Pipeline, Microarchitecture and System Design

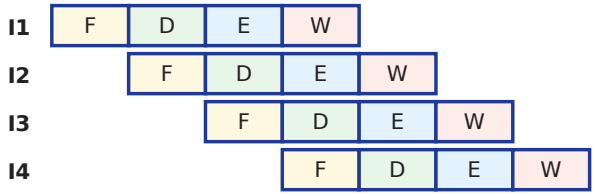
B4.1 Pipeline

- A technique that **improves the performance of the CPU**; instructions are processed **step by step**.
- A normal CPU (without pipeline) **completes one instruction and then starts the next**.
- In a pipeline, **while one instruction is being decoded, the next instruction can be fetched**.
- **Parallel execution** is allowed.
- With pipelining, **CPU speed increases and throughput also increases**.

Without pipeline — one instruction at a time (16 cycles)



With pipeline — stages overlap (7 cycles)



F = fetch · D = decode · E = execute · W = write back (illustration added)

Four instructions without and with a pipeline — the stages Fetch → Decode → Execute → Write back overlap (illustration added).

B4.2 Microarchitecture

- Shows **how the internal structure of the CPU is built** and how its components work internally.
- If system architecture is a **map**, then microarchitecture is the **detailed implementation of that map**.

Main components of microarchitecture

#	Component	Details
(i)	**Register**	The **fastest, smallest and costliest** memory of the computer; e.g. program counter, accumulator …; the in-built memory of the CPU
(ii)	**ALU**	Arithmetic and logic unit — performs the calculations and logical operations
(iii)	**Control unit**	Controls and manages all the devices and resources — which is why it is known as the **central nervous system**; decodes instructions and generates timing and control signals
(iv)	**Cache**	**Buffer memory between CPU and RAM**; manages the speed difference between CPU and RAM; 4 types: **L1, L2, L3, L4** (L1 is the fastest)
(v)	**Pipeline**	Technique to process multiple instructions at the same time: **Fetch → Decode → Execute → Write back**

The one-line description of the ALU is added; the notes list ALU only as a heading.

B4.3 System design architecture

- Shows the **arrangement of the system's major components** and the **data flow between them**.
- A **high-level design** that shows how the system will work and how its major parts are connected.

Level	Shows	Analogy
System design architecture	Major components and data flow — the big picture	The map
Microarchitecture	Internal structure of the CPU and how it works	Detailed implementation of the map

B5. Error Detection and Correction Codes

B5.1 Types of errors

Type	Meaning	Example
(1) Single-bit error	Only **one bit** gets changed	101011 → 101001
(2) Burst error	**Multiple bits** get changed	—

B5.2 Error detection codes

(i) Parity bit method (only to detect single-bit errors)

- An **extra bit (parity bit)** is transmitted along with the data.
- The **total number of 1s** is made either even or odd.

Type	Rule	Example from the notes
(a) Even parity	The number of 1s must be **even**	Data 101001 has three 1s → add parity bit 1 → **1010011** (four 1s, even)
(b) Odd parity	The number of 1s must be **odd**	If the 1s are already odd, the parity bit added is 0 (as in the notes' "… → 101000")

Example: Odd parity — both cases (added example)

Data	Number of 1s	Parity bit added	Sent
10110	3 (already odd)	0	101100
10100	2 (even)	1	101001

The handwritten odd-parity example is not fully consistent (10100 has two 1s), so the rule is shown with two clear examples.

(ii) Checksum method

- Mostly used in **network data transmission**.
- The data is divided into **equal-sized blocks / segments**.
- The **sum of all blocks** is generated using a checksum.
- This checksum is also **checked at the receiver end**.

Example: Data blocks 1001, 1100, 0101

Block	Binary	Decimal
1	1001	9
2	1100	12
3	0101	5
Sum	11010	26

Sum = 1001 + 1100 + 0101 = **1 1010**. The extra leading carry bit is **ignored** → **1010** is the checksum the sender sends. The receiver adds the received blocks again and checks the result against this checksum.

Clarification

The notes add the blocks and drop the carry. Real network checksums (e.g. in TCP/IP) usually add with end-around carry and send the one's complement, as the comparison table in B5.4 also says ("binary addition with end-around carry").

B5.3 Error correction code

- To **detect errors and automatically correct them, without data retransmission**.
- Used in more reliable systems like **memory and satellite communication**.

(i) Hamming code

- Proposed by **Richard Hamming**.
- Performs both **single-bit error detection and correction**.
- Some **redundant bits (r)** are added among the **data bits (d)**, so that each bit position can be represented by a unique binary number.
- To find the number of redundant bits:

Formula

$2^r \geq r + d + 1$

Example: 4 data bits

$2^r \geq r + 4 + 1 = r + 5$. Try r = 2: 4 ≥ 7? no. Try r = **3**: 8 ≥ 8? yes. So **r = 3** redundant bits (7-bit code: parity bits at positions 1, 2, 4).

Example: Worked Hamming example for 4 data bits (added — the notes stop at r = 3)

Data bits 1011 are placed at positions 3, 5, 6, 7; parity bits p1, p2, p4 sit at positions 1, 2, 4 (even parity).

Position	1	2	3	4	5	6	7
Role	p1	p2	d	p4	d	d	d
Bit	0	1	1	0	0	1	1

Parity bit	Checks positions	Bits	Parity bit chosen
p1	1, 3, 5, 7	d3 = 1, d5 = 0, d7 = 1 (two 1s)	0
p2	2, 3, 6, 7	d3 = 1, d6 = 1, d7 = 1 (three 1s)	1
p4	4, 5, 6, 7	d5 = 0, d6 = 1, d7 = 1 (two 1s)	0

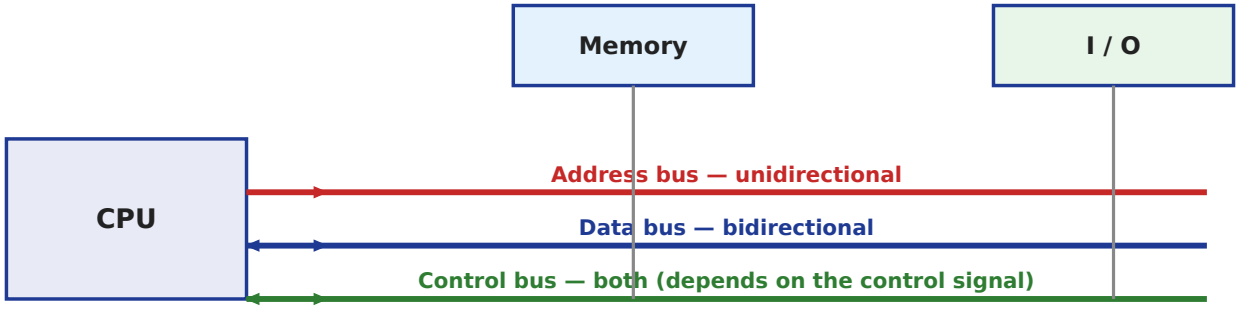
Code word = 0110011. If position 5 flips (0110111): p1 check fails, p2 check passes, p4 check fails → error position = p4 p2 p1 = 1 0 1 = 5 → flip bit 5 back. Single-bit error found **and corrected**.

B5.4 Comparison — parity vs checksum vs Hamming

Point	Parity bit	Checksum	Hamming code
Definition	Simple error detection by adding a single parity bit	Data blocks are added; in combination, a checksum is generated	Uses data + redundant parity bits for single-bit error correction
Main goal	Error detection	Error detection	Error detection + error correction
Category	Error detection code	Error detection code	Error correction code
Added redundant bits	Only 1 bit	16 or 32-bit checksum	Multiple parity bits (3, 4, 5 depending on data size)
Placement of extra bits	End of the data frame	After the data, as a checksum block	Parity bits at positions 1, 2, 4, 8
Technique used	Count 1s = even or odd	Binary addition with end-around carry	Parity bits calculated using multiple overlapped bit groups
Detects single-bit errors	Yes	Yes	Yes
Corrects single-bit errors	No	No	Yes
Detects double-bit errors	No	No (sometimes)	Yes — detect only, not correct (for more, Reed–Solomon & BCH codes) — as read
Reliability	Low	Medium	Very high
Use	Simple serial communication	TCP, UDP, IP, Ethernet	ECC RAM, satellite
Pros	Simple, cheap, fast	Detects large errors	Detection & correction — very accurate
Cons	Only detection	No correction	Complex and more overhead
Use case	Low risk	Networking protocols	High-reliability systems

B6. Bus

- A **communication path / set of wires / set of lines** through which, in a computer system, **data, address and control signals** are exchanged between multiple parts.



address: where · data: what · control: read / write / interrupt

The three buses between the CPU, memory and I/O (illustration added from the descriptions in the notes).

Bus	Direction	Carries	Example from the notes
(i) Address bus	Unidirectional	The **address of the memory location** where data is to be read / written. When the CPU reads data from memory it sends that address through the address bus	A 16-bit address bus can access 2^{16} = 65,536 (64 KB) memory locations
(ii) Data bus	Bidirectional	The **actual data** being transferred between the CPU, memory and I/O. CPU → memory = data write; CPU ← memory = data read	A data bus width of 16 bits carries 2 bytes of data per cycle
(iii) Control bus	Both (based on the control signal)	**Control and timing signals**; indicates which type of operation is being performed (read / write / interrupt)	—

B7. Microprocessor and Microcontroller

B7.1 Microprocessor

- Invented in **1971**: the **Intel 4004** by Intel — the first, smallest and complete CPU (**4 bits**).
- After 1971 came the Intel 8008, Intel 80386 etc.; today the Intel i-series (Itanium) and AMD Ryzen series — complex, multitasking CPUs — are available.
- **Use:** personal computers, laptops, servers, gaming consoles (PlayStation and Xbox).
- **Intel x86** came in **1978** (based on the Intel 8086 architecture).

Type of microprocessor	Meaning
(a) RISC	Reduced instruction set computing
(b) CISC	Complex instruction set computing
(c) VLIW	Very Long Instruction Word — parallel processing
(d) EPIC	Explicitly Parallel Instruction Computing — advanced VLIW

B7.2 Microcontroller

- Along with the microprocessor, **memory (RAM & ROM)** and **peripherals (input–output & timers)** are also used, so it is mainly used in **embedded systems**.
- The first microcontroller = **Intel 8051 (1980)**.
- After this, **PIC, ARM, AVR** are popular microcontrollers.
- **Use:** embedded systems — microwave oven, washing machine, robotics, automobiles, medical devices.

Type	Full form	Developed by	Details / use
(a) PIC	Peripheral Interface Controller	Microchip Technology	8, 16 and 32-bit variants; consumer electronics, embedded systems and automation
(b) AVR	Advanced Virtual RISC	Atmel (now Microchip)	Based on RISC; used in small embedded devices
(c) ARM	Advanced RISC Machine	—	RISC-based powerful microcontroller family; well known for **low power consumption yet high performance**; widely used in smartphones and embedded systems

	Microprocessor	Microcontroller
Contains	CPU only (memory and I/O are separate)	CPU + memory (RAM & ROM) + peripherals (I/O, timers)
Mainly used in	PCs, laptops, servers, gaming consoles	Embedded systems
First one	Intel 4004 (1971)	Intel 8051 (1980)
Examples	Intel i-series, AMD Ryzen	PIC, AVR, ARM

Comparison table added for revision from the two sections above.

Appendix — Clarifications and reading notes

The handwritten pages were typed as faithfully as possible. These places were clarified, flagged or marked as added:

Topic	In the handwritten notes	Here
TOC — innermost class of the Chomsky picture	“Recursive language” next to Type 3 / FSA	Shown as regular language
TOC — DFA diagram	Label of the arrow A → B illegible	Read as “a” (accepts strings starting with a); flagged
TOC — DFA vs NFA table	“Each NFA can’t be a DFA”; backtracking possible for DFA	Kept as written; standard statements added in a note
TOC — Σ* notation	$\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$	Kept; $\Sigma^+ = \Sigma^*$ minus ε
COA — organization vs architecture table	Second row heading hard to read	Marked as “logic / level”; flagged
COA — logical instructions	“AND, OR, AND”	Read as AND, OR, NOT
COA — odd parity example	Data 10100 labelled “1s already odd”	Rule restated with two correct examples
COA — checksum	Sum with carry ignored	Kept; one’s-complement / end-around-carry note added
COA — Hamming double-bit cell	Remark about Reed–Solomon & BCH hard to read	Transcribed as read; flagged
Added for revision	—	Prefix/suffix lists, Σ^n table, precedence example, machine / architecture comparison tables, Hamming worked example, pipeline, von Neumann, Harvard and bus pictures